

SurakshaNet: on-device abuse detection for Hinglish chat

B.Tech mid-semester briefing · 15 minutes

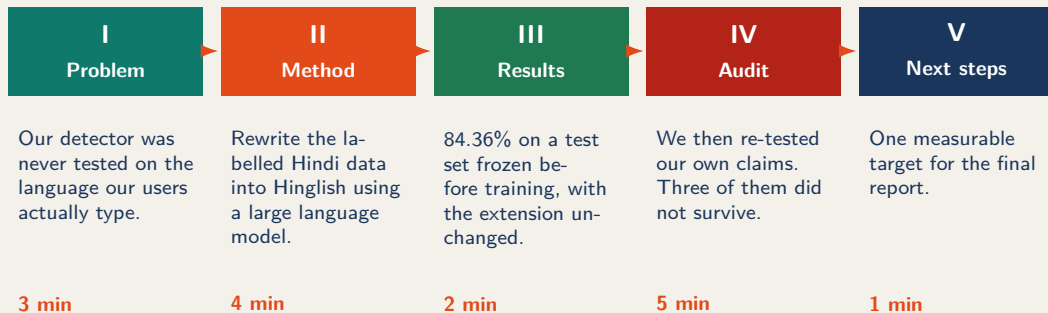
Aamod Jain and Pratham Garg · 20 September 2026

WHAT SHIPS: A BROWSER EXTENSION, NOTHING ELSE



No message ever leaves the device. The contribution is training data, not a new inference path.

Where this talk is going



Part IV is the substance of this briefing. We spent this iteration testing our own Part III numbers rather than adding new ones, and it changed what we are willing to claim.

Six terms used throughout this talk

Hinglish

Hindi grammar, written in Roman letters, with English words mixed in. This is what people actually type.

Code-mixing index (CMI)

How deeply two languages are mixed in one sentence. 0 = only one language. Humans average ≈ 36 .

MACD

Our labelled dataset: Hindi comments in Devanagari script. Label 0 = abusive, 1 = not abusive.

MiniLM INT8

The small model that runs inside Chrome. “INT8” = compressed to 8-bit so it fits in a browser.

Recall vs specificity

Recall: share of abusive messages we catch. *Specificity*: share of clean messages we correctly leave alone.

p -value

Chance of seeing a difference this large if there were really no difference. Below 0.05 we call it real.

ONE SENTENCE, THE TWO FORMS THAT MATTER — both mean “there is a meeting at 4 tomorrow”

What our labelled data looks like: Devanagari Hindi कल मीटिंग है चार बजे

What a real user types: Roman Hinglish kal meeting hai 4 baje

PART I

Problem

Why an abuse detector that scores 84% can still be untested on real traffic.

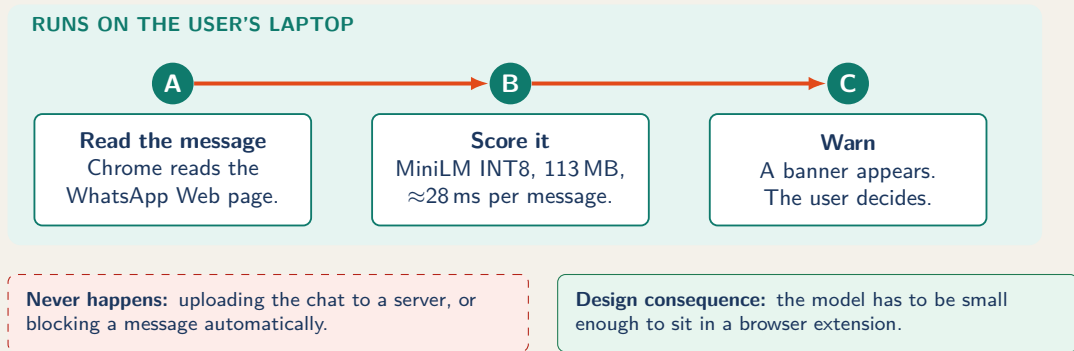
Questions this part answers

What ships today? A Chrome extension that scores WhatsApp Web messages entirely on the user's laptop.

What was measured? Hindi in Devanagari script, and English. Never the Roman Hinglish people actually type.

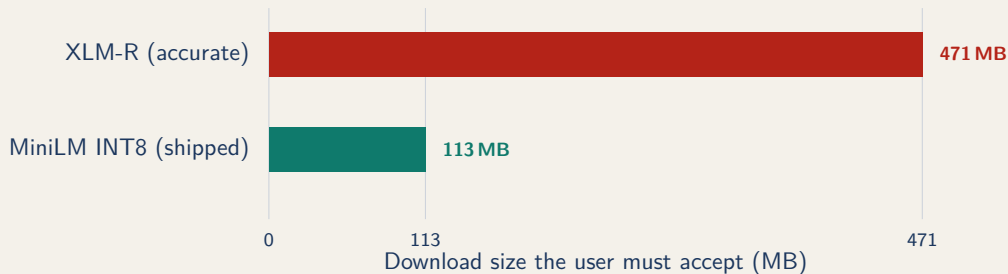
Why not just collect Hinglish data? No labelled Hinglish abuse dataset exists for this task.

What we built: a warning, not a filter, and it never uploads



Why this matters for the rest of the talk: every decision that follows is constrained by a model that must fit on a laptop and answer in milliseconds.

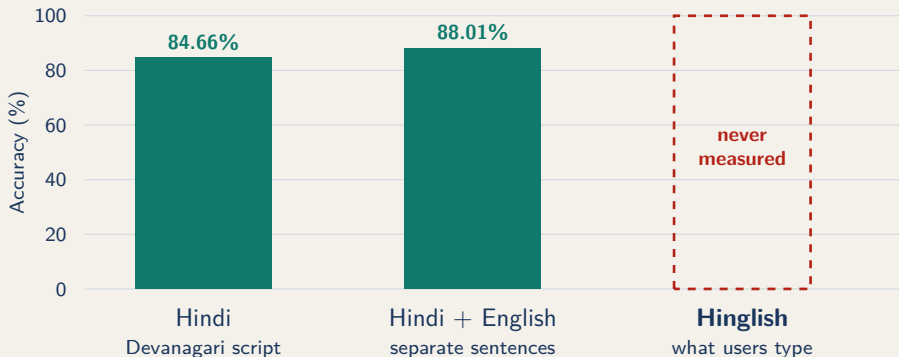
The accurate model is four times too large to ship



What shrinking costs us. On the published Hindi benchmark, XLM-R reaches 86.34% and our small model reaches 84.66% — a gap of under 0.11 pp once quantised.

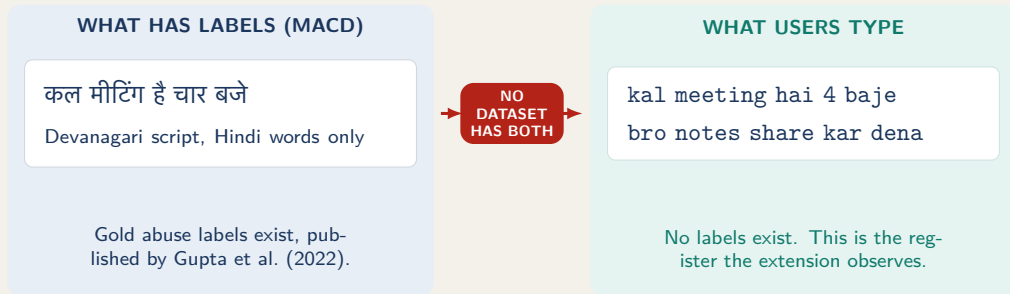
So the small model is not the problem. The problem, on the next slide, is which *language* we ever tested it on.

The gap: we never tested the language our users type



The dashed box is not a low score — it is a *missing experiment*. Our earlier poster reported the two solid bars and stopped there. A detector deployed on WhatsApp sees almost none of either: it sees `kal meeting hai 4 baje`, which is neither of the two things we tested.

Why we could not simply download Hinglish training data



And the standard trick does not apply. The usual way to build code-mixed text (WAC and PAC) needs each sentence paired with its English translation. MACD ships Hindi only, with no English pair, so those methods are unusable here. That is what forced us toward a language model.

The research question, and the three rules we set ourselves

Can we rewrite labelled Hindi abuse data into natural Hinglish, keep the original labels honest, and prove it on a test set frozen before training?

Rule 1 — don't touch the product

The extension's read-score-warn path stays exactly as it is.

Rule 2 — don't launder the labels

A message labelled abusive must still be abusive after rewriting.

Rule 3 — don't move the goalposts

The 6368-message Hinglish test set is frozen before any training.

Rule 2 is the one that later turns out to be hard to verify, and Part IV is mostly about that.

PART II

Method

Turning Devanagari Hindi into Hinglish without inventing new labels.

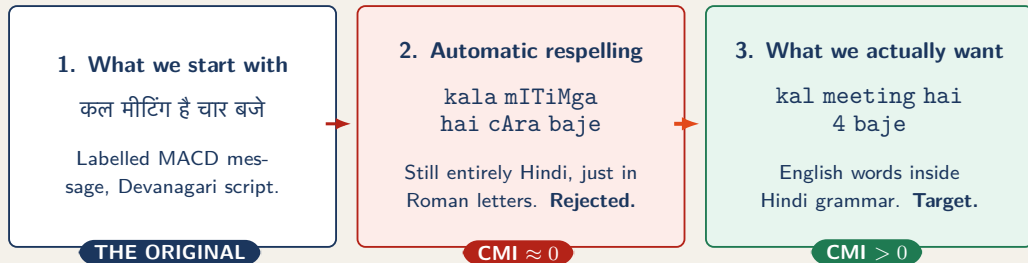
Questions this part answers

Can't we just respell it in Roman letters? We tried. Respelling is not mixing, and we show why it fails.

So how do we get Hinglish? An LLM rewrites each message once, offline. It never runs inside the extension.

How do we know the rewrite is still abusive? A two-part gate keeps or quarantines each rewrite — this is the part we later audit.

First attempt: respelling Hindi in Roman letters is not mixing



Respelling was the cheap option — no GPU, no cost. But the code-mixing index stayed at zero, which is the measurement telling us the sentence is still monolingual Hindi. **Changing the alphabet is not changing the language**, so a model trained on column 2 would still never see an English word.

Our method: one offline rewrite pass, outside the product

WHAT THE USER RUNS — no language model here

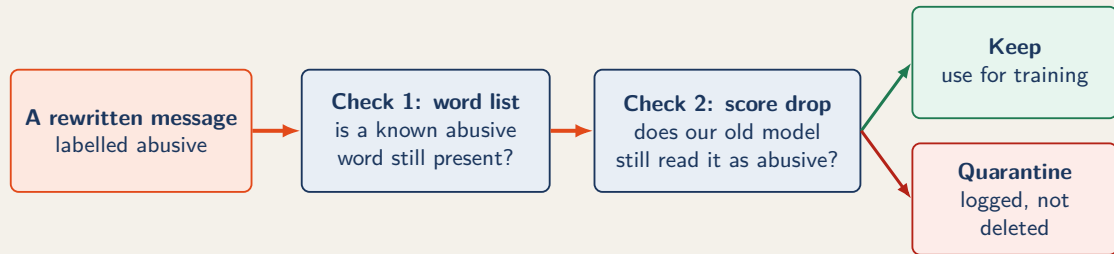


WHAT WE RAN ONCE, ON A SERVER — training data only



31 663 messages were rewritten once through Qwen3-32B on AWS Bedrock. The language model is a *data converter*, not a detector: it is never downloaded, never called at inference, and cannot affect a user's latency or privacy. Only its output — rewritten training rows — reaches the shipped model.

Keeping the labels honest: the gate, and how it is supposed to work



The worry: a language model asked to rewrite an insult might quietly *soften* it. The message would then sit in training data labelled “abusive” while no longer being abusive — and we would be teaching the detector something false.

Check 2 in plain terms: we score the Hindi original and the Hinglish rewrite with our previous detector. If the rewrite’s abuse score falls by more than 0.15, we assume abuse was lost and quarantine the row. Written out: $p_{\text{rewrite}} \geq p_{\text{original}} - 0.15$.

Freezing the test set before training, so the result means something

Allowed

- Cleaning or re-labelling the *training* rows.
- Quarantining a rewrite, as long as it is logged rather than deleted.
- Reporting the code-mixing index even when it is embarrassingly low.

Forbidden

- Editing the 6 368 frozen test messages after seeing our errors.
- Shipping whichever variant happened to score best.
- Putting the language model inside Chrome.

A test set chosen *after* looking at mistakes will flatter any model, and the number stops being comparable to the published baseline. So we fixed the 6 368 Hinglish messages first and did not touch them again.

Worth remembering for Part IV: we wrote the second forbidden rule down, and then broke it.

PART III

Results

What the retrained detector scores, and how human-like our generated Hinglish really is.

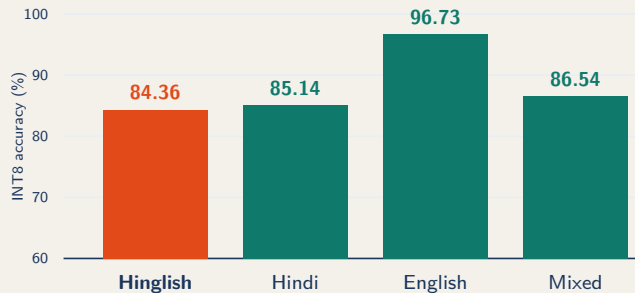
Questions this part answers

Does it work on Hinglish now? Yes — 84.36% on the frozen test set, with the extension untouched.

Did we break anything else? Almost: dropping the original Hindi data collapsed Hindi accuracy to 70.04%.

Is the generated Hinglish realistic? Not yet. It mixes far more shallowly than humans do, and we report that openly.

Result: the missing cell is filled, at 84.36%



The product did not change.

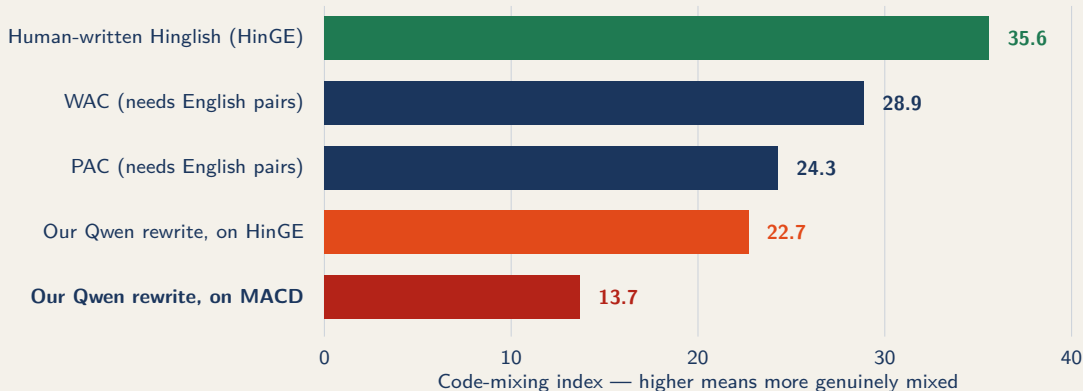
Same MiniLM, same 113 MB, same ≈ 28 ms. Only the training data changed.

One near-miss worth reporting.

Training on Hinglish *instead of* Hindi pushed Hindi accuracy down to **70.04%**. We put the Hindi data back and it recovered to 85.14%.

Frozen Hinglish test, $n = 6368$, never seen during training. “Mixed” is all three languages together; it reads 86.54% rather than the earlier 88.01% precisely because Hinglish is now inside the mix and Hinglish is the hardest of the four. **A Hinglish gain paid for with a Hindi loss is not a gain**, so that variant was rejected.

But our generated Hinglish mixes far more shallowly than humans



Every automatic method falls short of humans, but ours falls shortest on the data we actually train on: **13.7 against 35.6**, and **25%** of our rewrites contain no mixing at all (CMI = 0). On translation-style quality our rewrites are competitive (BLEU 17.7 versus WAC 16.4 and PAC 17.9) — the shortfall is specifically in *depth of mixing*, not fluency. This slide is what made us go back and audit everything in Part IV.

PART IV

Audit

We re-tested our own Part III claims against the files on disk. Three did not survive.

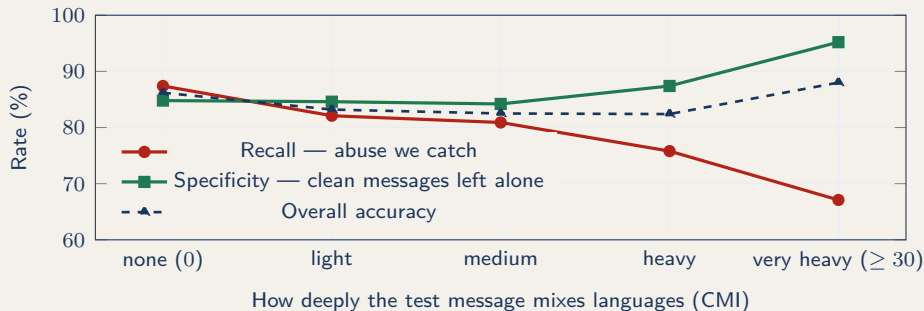
Questions this part answers

Is 84.36% a meaningful summary? It hides a failure that gets worse exactly where mixing gets deeper.

Did the label gate actually work? It was never tested. When we finally tested it, it barely beats chance on most rows.

Were our ship / don't-ship calls sound? One of the two was not statistically supported and is withdrawn.

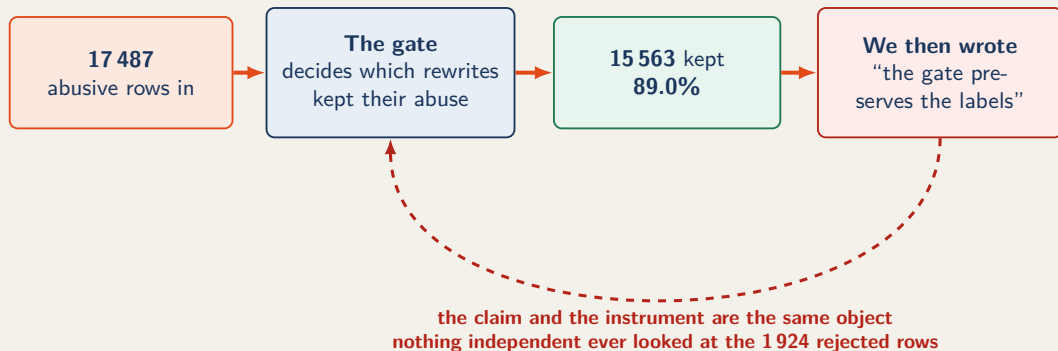
Finding 1: we catch less abuse exactly where mixing is deepest



Read the red line. On unmixed messages we catch 87.4% of abuse; on the most heavily mixed we catch 67.1%. The trend is real, not noise ($p = 5.5 \times 10^{-13}$).

Why one accuracy number hid this. Heavily mixed messages are mostly *clean* — the abusive share drops from 54.7% to 25.6% — so a model that drifts toward saying “not abusive” is rewarded there. Accuracy (dashed) even rises. The single 84.36% headline is therefore the wrong summary for a safety feature.

Finding 2: our evidence that labels survived was the gate marking its own work



Two further problems with that number. It was presented as a result of the *word list*, but of the 1 924 rejections the word list made only **74** — the score-drop check made **1 806**. And the 0.15 threshold in that check was never derived: it is a default value in a script, reused unchanged for a second, different purpose. There is no threshold sweep and no human-labelled check anywhere in the repository.

So we built something to test the gate against. Twice it was wrong

Attempt 1 — invalid

Auto-build a slur list, delete one slur, call that “abuse removed”. But 24% of those rows still contained a slur, and harmless words like *kapde* (clothes) were deleted.



Attempt 2 — circular

Hand-pick unambiguous slurs and verify the deletion. Labels now sound, but deleting a word leaves broken grammar, and “is the word gone?” is exactly what the word list already checks.

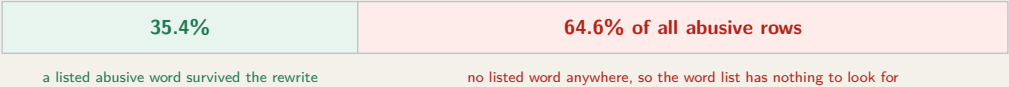


Attempt 3 — used

Ask the same model to rewrite each message twice: once faithfully, once deliberately softened. Both versions are fluent, so no checker can win just by spotting bad grammar.

This is the part we would have skipped a month ago: you cannot score a detector without something to score it *against*. Building that reference took 5 266 graded message pairs and roughly 2 100 model calls. **The honest limitation:** the “softened” label comes from the instruction we gave the model, checked only by word match. Having humans certify a sample is the first task of the next iteration.

Measured against that reference, the gate is blind on most rows



Checker	Catches real softening	Wrongly rejects faithful rewrites
Word list, on the 35.4%	99.8%	0.0%
Word list, on the other 64.6%	0.0%	0.0%
Score drop > 0.15 (we shipped this)	43.8%	24.9%
Score drop, best threshold	52.8%	32.2%
An LLM shown the original	27.0%	1.3%

Where the word list applies it is near-perfect and the score check only adds false rejections — so **for a third of the data a word list really is enough**. On the other two thirds the word list returns “fine” because it was never able to look, leaving the score check alone at an AUC of **0.629**, where pure chance is 0.500. Re-tuning the threshold did not help.

Finding 3: one of our two “do not ship” verdicts does not hold

What we compared	Shipped	Alternative	p -value	Verdict now
shipped vs extra data cleaning	84.28%	83.81%	0.179	cannot tell them apart
shipped vs cleaning + synthetic rows	84.28%	83.46%	0.027	genuinely worse

We chose the model using the frozen test

The training script picks the best variant by its *test* accuracy and promotes it. That is the rule we wrote down as forbidden on slide 14, so 84.36% is optimistic by an unknown amount.

Neither artifact can be rebuilt

The shipped model’s recipe matches no configuration in the current script, and promoting it overwrote the very checkpoint the label gate had used for scoring.

Re-running the three models reproduces their logged scores to within 0.18 pp — about the same size as the differences we were drawing conclusions from. Small gaps need a paired test, not an eyeball.

Two claims we made during this audit are also withdrawn

Withdrawn: “the gate punishes code-mixing”

On the invalid first reference this looked significant. On the corrected one, adding English words raises mixing from 10.0 to 27.0 and rejection actually *falls* ($p = 0.19$). The effect runs the opposite way.

Withdrawn: “about 14% of labels are wrong”

That figure came from inverting the 11% rejection rate with a formula that assumed one attempt per message. The pipeline retries, so the arithmetic does not apply.

What we say instead. Because we now know what fraction of softening the gate catches, we can state contamination as a range rather than a single number: if one rewrite in five is softened, the gate brings bad labels down from 20% to 15.8%. It removes roughly a fifth of what it exists to remove, while discarding 1 924 abusive rows to do it. We report that as a range because the softening rate itself is still unmeasured — and measuring it needs humans.

Conclusion: what stands, and the one number to beat

What stands

- The product is unchanged and works: on-device, 113 MB, ≈ 28 ms.
- Hinglish went from unmeasured to measured, and Hindi was not sacrificed for it.
- The real defect is mixing *depth*, not Hinglish itself.
- We now know what our label gate can and cannot do, which we did not before.

Next iteration, in order

- A** Have humans label 400 rewrites and report agreement — every automatic checker failed, so this is unavoidable.
- B** Rebuild the gate per stratum: word list where it works, an LLM judge where it does not.
- C** Generate training data balanced across mixing depth.
- D** Re-freeze a depth-stratified test set; use a paired test for every decision.

The target for the final report. Catch **82%** of abuse in heavily mixed messages, up from **72.4%** today, without letting clean-message accuracy fall below 84%.